ADS-B / AIRCRAFT TRACKING / SELF-HOSTED

Aerodrome

A clean, modern ADS-B aircraft tracker for your home receiver.

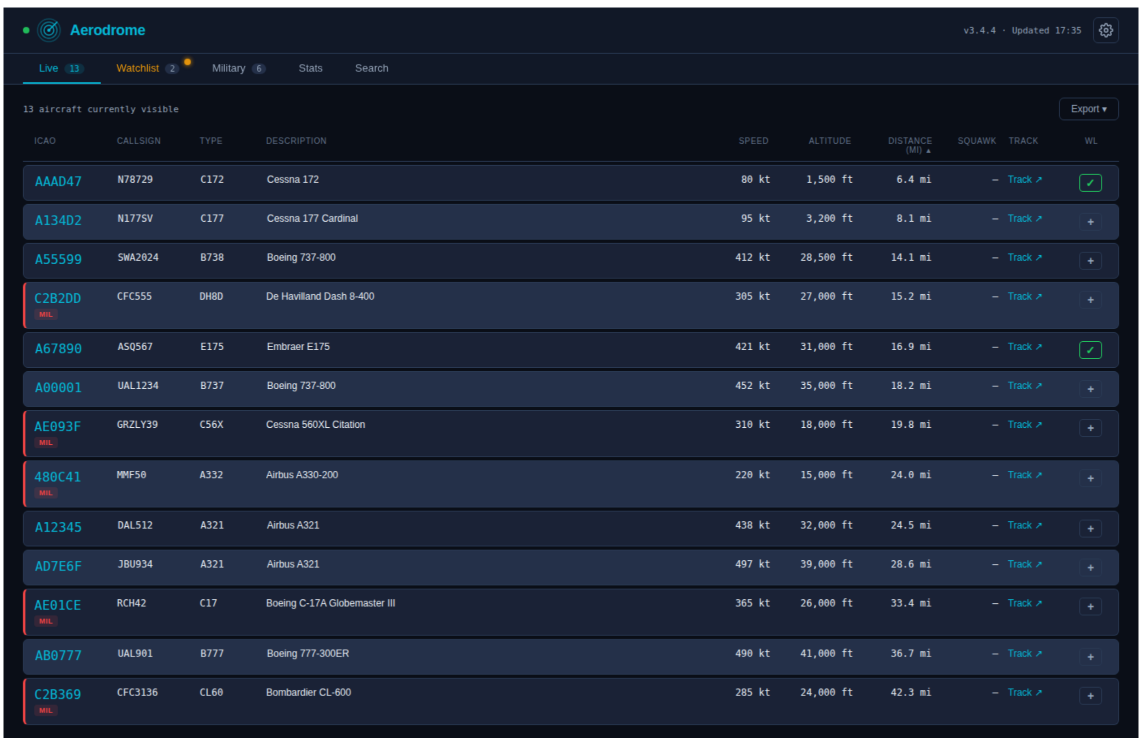
Turn your local ADS-B receiver (readsb, dump1090, tar1090) into a polished web dashboard: live aircraft, a personal watchlist, auto-detected military flights, a searchable history, and a rich statistics view — with optional push notifications to your phone.

What Aerodrome is

Aerodrome is a self-hosted web dashboard for your personal ADS-B receiver. Point it at readsb, dump1090, tar1090, or any ADS-B source that exposes a JSON aircraft feed, and it turns the raw stream into something you'd actually want to leave open on a second monitor.

It's not a replacement for FlightAware or the giant commercial trackers. Those show you the whole sky; Aerodrome shows you *your* sky — whatever your antenna can hear, indexed and searchable, with a watchlist for aircraft you care about, automatic highlighting of military traffic, a rich statistics view, and push notifications to your phone when something interesting appears overhead.

It's open-source, runs on anything from a Raspberry Pi to a spare x86 box, stays entirely on your LAN unless you choose otherwise, and has no cloud dependencies beyond an optional registration-lookup API.



The Live view — aircraft currently visible to the receiver.

A brief history

Aerodrome started life as a bash script — `adsbmilitary.sh`, version 1.6.6. It did one useful thing: polled a local readsb instance on a cron schedule, parsed the output with `jq`, and sent a Pushover notification whenever a military aircraft appeared overhead. Functional, unpretentious, and written the way every useful tool starts.

The decision to rewrite came out of two realizations. First, the bash script had outgrown bash — every new feature meant wrestling with shell quoting and cron behavior. Second, once you have a script that's been running at home for a year, you start wanting to actually look at the data. Not just notifications when a C-17 flies over, but questions like: what's the busiest hour of my airspace? When did I last see this tail number? How far out can my antenna reach?

The rewrite became a structured Python application: a polling collector, a FastAPI web server, a SQLite database in WAL mode, a dark-themed single-page frontend, and a configurable notification system. The name "Overwatch" was considered and rejected (already a video game); "Aerodrome" stuck.

The project has gone through a lot of iteration since then. As of this document, the changelog has **373** numbered releases. Most of them are small — a new stat card, a tooltip, a bug fix — but a handful are substantial: the rich Stats tab with coverage roses and all-time records, the optional in-house ntfy push notification server, the full backup and restore flow, the upload-and-apply updates from the web UI, and most recently a performance diagnostic page for people running the tracker on constrained hardware at serious scale.

The throughline has been "build the tool you want to use yourself, not the product you'd pitch." Every feature exists because someone — usually the author, sometimes a user running on a Pi near a busy airport — hit a specific frustration and asked for a fix.

What it does

The main tracker interface has five tabs across the top.

Live. what's in the sky right now. Refreshes every few seconds. ICAO, callsign, type, altitude, speed, distance. Military aircraft are highlighted; watchlist entries are tagged. Click any aircraft to see an expanded drill-down with a tracking link to an external map.

Watchlist. aircraft you've flagged to follow. Add by ICAO, callsign prefix, or a substring match on aircraft type (so "cirrus" catches every Cirrus). Tabs badge when there's a watchlist aircraft currently in the air.

Military. auto-detected military flights based on hex-range rules. Shows every military aircraft seen in your retention window, with special-category labels (AWACS, refueling, VIP transport, etc.) where applicable.

Stats. the rich analytics tab. Today's counts, all-time records, aircraft-type breakdown, hourly activity histogram, "first time seen" list, 7-day unique-aircraft chart, watchlist frequency, and receiver coverage rose showing which compass directions your antenna pulls best.

Search. full-text search across every aircraft your receiver has ever seen, going back as far as your retention window allows. Type a callsign, ICAO, type, country, operator, or relative-date token like `today`, `hour:14`, `distance:50-100`, `military`, `watchlist`. Sortable columns, date-range presets, page-size control, CSV export, and shareable hash URLs. Stats drill-panels deep-link here with the right filters and sort already applied.

The Stats tab — long, because there's a lot to show.

How it works

Architecturally, Aerodrome is three moving parts in one Python process.

The collector. polls your ADS-B receiver every 60 seconds (configurable), pulls the current aircraft JSON, classifies each hex (normal / military / watchlist), enriches with registration lookup via `hexdb.io`, and appends rows to a SQLite database. Uses Write-Ahead Logging mode so reads and writes don't contend.

The web server. a FastAPI application that serves the dashboard templates and the 88 JSON API endpoints behind them. Single-process, bound by default to your LAN IP on port 8000.

The notifier. optional. Dispatches push notifications via ntfy (self-hosted or the public ntfy.sh relay) when aircraft matching your rules appear. Includes cooldown and rate-limit logic so a single busy hour doesn't spam you with alerts.

Data model

11 tables. Three are sighting logs with configurable per-table retention (`all_sightings`, `military_sightings`, `watchlist_sightings`) — they get pruned on a schedule. Two are permanent: `seen_aircraft` stores the first-ever sighting of every ICAO, and `stats_records` holds the all-time superlatives (furthest, fastest, highest, etc.). Everything lives in one SQLite file that's straightforward to back up, inspect, or copy to a bigger disk.

Runs as a systemd service

Install produces a proper systemd unit that starts on boot, restarts on failure, and logs to `journalctl`. The included install script handles Python venv creation, dependency install, and a scoped `/etc/sudoers.d/aerodrome` rule that lets the web UI perform specific privileged operations (service restart, ntfy install) without giving the service account blanket root access.

Push notifications

Aerodrome pushes alerts to your phone via **ntfy** — an open-source pub/sub notification service by Philipp C. Heckel. You can point Aerodrome at the public ntfy.sh instance (free, anonymous, easy) or install a private ntfy server locally — Aerodrome will do that for you with one click from the Notifications settings page.

Watchlist hits. an aircraft matching one of your watchlist rules appears in range.

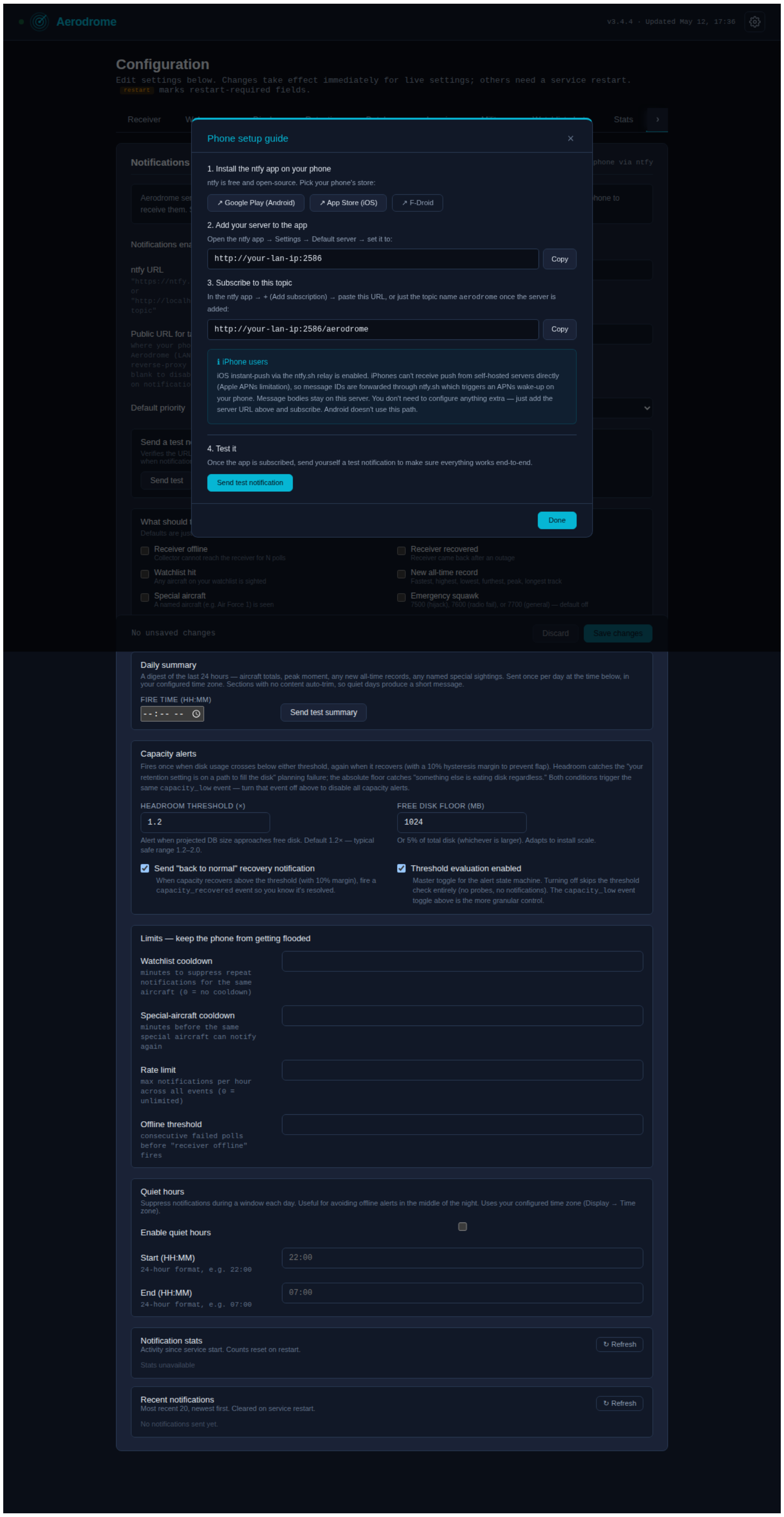
Military overhead. any military aircraft enters your reception radius.

Special categories. AWACS, tankers, VIP transports, and other specifically-tagged flights.

Receiver offline. your local ADS-B feed has gone silent — often the first sign a receiver or antenna has a problem.

Daily summary. a once-a-day recap of activity: unique aircraft, military count, watchlist hits, furthest contact.

Each event type can be enabled or disabled independently. Cooldown rules prevent the same aircraft from paging you every time it re-enters range. An hourly rate limit protects against notification storms if the watchlist is too broad. A built-in setup wizard walks users through installing the ntfy mobile app, adding the server URL, subscribing to the topic, and sending a test notification to verify the full path works.



The onboarding modal — mobile setup in four steps.

Performance at scale

ADS-B traffic volume varies dramatically depending on where your antenna lives. A rural receiver might log a few hundred sightings a day; one near a major airport can log millions. Hardware requirements scale accordingly.

Hardware tiers

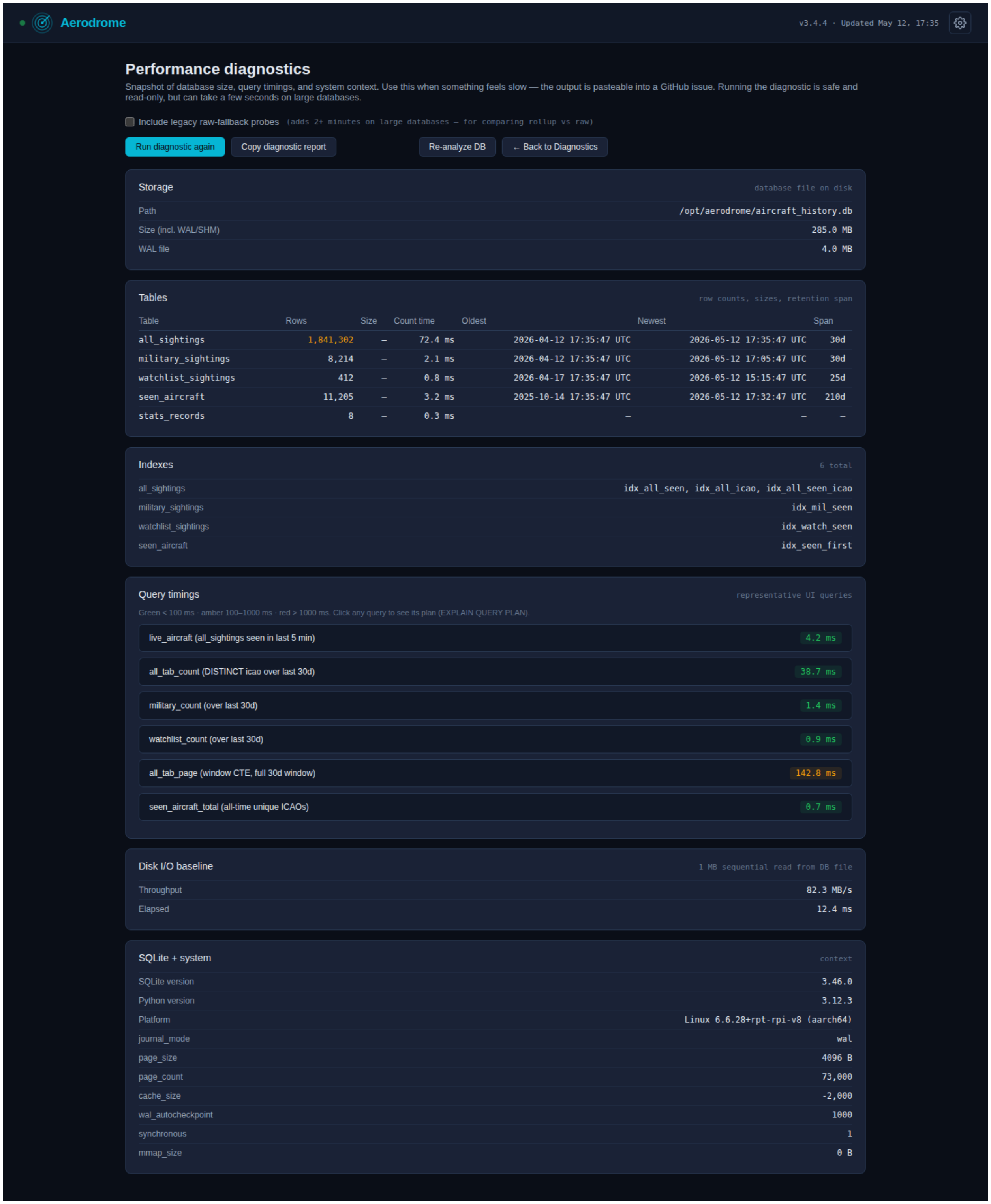
Rural / quiet airspace. Under ~500 aircraft/day, under ~300k sightings at 30-day retention. Raspberry Pi 4 with any reasonable SD card is fine. Everything is fast.

Suburban / moderate. ~500-2,000 aircraft/day, ~300k-1.5M sightings at 30-day retention. Raspberry Pi 4 still works, but use an A2-rated SD card for random-I/O performance (examples as of 2026: SanDisk Extreme Pro, Samsung Pro Plus). Stats tab loads in the 3-8 second range.

Major-airport / heavy. 2,000+ aircraft/day, 1.5M+ sightings at 30-day retention. Raspberry Pi 5 strongly recommended, and an NVMe HAT is worth the money — the Stats tab's worst-case queries are bound by storage I/O, and NVMe is roughly 10-20x the random-read throughput of an SD card. Example HATs (as of 2026): Pimoroni NVMe Base, Pineberry Pi HatDrive. Any small x86 box with an SSD is an equivalent alternative.

If you don't want to upgrade hardware. Reducing retention is the easiest win. A 3M-row install at 30-day retention becomes a 1M-row install at 10-day retention — and often runs comfortably on the original hardware tier. Adjust in the Configuration → Retention tab in the web UI, or edit `retention.all_days` in `config.yaml`.

Aerodrome ships a built-in **Performance diagnostic page** that measures your actual system: database size and per-table row counts, SQLite pragmas, index coverage, six representative query timings with `EXPLAIN QUERY PLAN` output, disk-read throughput, and auto-generated hints. It produces a plaintext report that captures the whole picture at once. No guessing — measurement first.



The Performance diagnostic page — storage, query plans, and auto-generated hints for constrained hardware.

Operations

Running a service at home long-term means updates, backups, and things occasionally going wrong. Aerodrome takes this seriously.

Updates via the web UI

The Updates page polls GitHub on a configurable cadence (daily, weekly, monthly, or never) and shows when a new release is available. Click "Apply update" and the server downloads the release zip, verifies its SHA256 checksum, backs up the current install, swaps in the new files, re-installs dependencies, and restarts. The old install stays in `.backups/<timestamp>/` in case you need to roll back. Three optional notification surfaces — in-card banner, gear-menu badge, and ntfy push to your phone — let you know about new releases without checking the page. Dropping a local zip onto the Updates page and SSH/rsync both still work as alternatives for specific-version installs or scripted workflows.

Full backup and restore

One click downloads a complete disaster-recovery zip: config.yaml, the aircraft-history database (snapshotted safely via SQLite's online backup API so it's consistent even while the service is writing to it), and the ntfy server config if you're running one. Upload to a fresh Aerodrome install and you're back to where you were.

Health monitoring

A gear menu badge lights up amber when something needs attention — sudoers drift after an update, sustained system load, a degraded hexdb resolver, a core component offline. The Status page breaks down every subsystem: collector thread, web server, database connectivity, receiver reachability, tail-number resolver latency, plus system-level CPU/load/memory/disk with color coding.

It honestly doesn't need much babysitting

The tracker has been running continuously at the author's home for many months across dozens of releases. Most weeks, the only maintenance is checking the Stats tab to see what interesting flew over.

Fun stats

Some numbers about the project itself, for the curious.

373 numbered releases in the changelog	**16,006** lines of Python across 6 modules	**21,991** lines of HTML/JS across 12 pages
88 JSON API endpoints under /api/	**11** SQLite tables, WAL mode	**1** bash-script ancestor (adsbmilitary.sh 1.6.6)

And some things it does *not* have:

No cloud account. No user tracking. No telemetry. No ads. No subscription. No analytics SDK. No "premium features." No agreement to sign. It runs on your hardware, serves your data, and talks to exactly the services you tell it to.

What's interesting about the data

On a moderate suburban receiver, you'll typically see between 500 and 2,000 unique aircraft per day — a mix of airliners, general aviation, the occasional helicopter, and whatever military traffic is routing through your airspace that week. The Stats tab will surface patterns you wouldn't have noticed otherwise: which airline dominates your skies, the shape of the morning rush, the range of your antenna in each compass direction, and the rare long-tail sightings (a research aircraft, a one-off military exercise, a transatlantic diversion).

Getting started

What you need

An ADS-B receiver on your network. anything that exposes a JSON aircraft feed — readsb, dump1090-fa, tar1090, PiAware, etc. Most receivers run this by default on port 8080.

A host to run Aerodrome. Ubuntu 22.04+ or Debian 12+ recommended. Hardware depends on traffic volume — see the *Performance at scale* section above for concrete tier guidance. Python 3.10+.

(Optional) a phone. for push notifications — both iOS and Android are supported via the ntfy mobile app.

Installation

One command on a fresh Ubuntu 22.04+ or Debian 12+ host: `bash <(curl -fsSL https://install.aerodromeadsb.com)`. The bootstrap detects your platform, installs prerequisites, downloads the latest release with SHA256 verification, prompts for the bare minimum config (receiver IP, optional latitude/longitude), and hands off to the bundled install script which creates a Python virtualenv, writes the systemd unit, sets up the scoped sudoers rule, and starts the service. Open `http://your-host:8000/` and visit the gear menu's Configuration page to adjust settings — auto-detected timezone, watchlist, notifications, retention, display preferences. The manual install path (download zip, edit `config.yaml`, run `./install.sh`) remains supported for offline installs, version-pinning, and git-checkout workflows.

Don't have a real ADS-B receiver yet? Add `--demo` to the install command and Aerodrome installs in demo mode with a small synthetic feeder running alongside it — the dashboard fills with 50 simulated aircraft, a starter watchlist, occasional military traffic and emergency squawks. An in-app wizard at Configuration → Demo handles the transition to your real receiver when you're ready. See the *Demo mode* section in `docs/INSTALL.md` for details.

The first minute of data will populate as the collector polls the receiver. Watchlist and notifications can be configured through the web UI — no YAML editing required after initial setup.

Tech stack

Language. Python 3.10+, a little JavaScript (vanilla, no build step, no framework).

Web. FastAPI + Uvicorn. Jinja templates served as static HTML.

Database. SQLite in WAL mode. No external DB server.

Push. ntfy, self-hosted or public. Installed and managed by Aerodrome itself if you want the self-hosted path.

Deployment. Systemd service. Install scripts for Ubuntu/Debian.

License. MIT. Use it, fork it, modify it, redistribute it.